



**Lab Title:** HTML Injection

**Student Name:** Nebeuwa Ifeanyichukwu Raphael

**Student ID:** 202520850LE

**Course Name:** BVWS103 – Owasp Top 10 Vulnerabilities  
Exploitation Techniques

**Instructor Name:** AMMAR AMINU

**Date of Submission:** 25/01/2026

## Executive Summary

This lab focused on understanding HTML injection and how it affects web applications. I learned how user input can be misused to inject and run unwanted HTML or JavaScript code when proper validation is not applied. I also practiced fixing this issue by sanitizing user input to prevent malicious code from executing. The lab helped me understand why input validation is very important in web security.

## Lab Objective

The objectives of this lab were to:

- Understand what HTML injection is and how it works.
- Learn how attackers can exploit HTML injection using user input.
- Practice preventing HTML injection by sanitizing input using secure coding methods.

## Lab Environment

The lab was carried out in a local testing environment using:

- A web browser to access the application.
- PHP for server-side scripting.
- HTML for building the user input form.
- A local development setup to safely test both vulnerable and secured code.

This controlled environment allowed me to test the vulnerability and apply security fixes without any risk to real systems.

## Task 1: Exploit HTML Injection Vulnerability

### Step 1: Setting up my Vulnerable Web Application

1. Created my new folder (Lab2\_HTMLInjection)

```

(raph@kali)-[~]
$ sudo mkdir /opt/lampp/htdocs/Lab2_HTMLInjection

(raph@kali)-[~]
$ cd /opt/lampp/htdocs
cd: no such file or directory: /opt/lampp/htdocs

(raph@kali)-[~]
$ cd /opt/lampp/htdocs

(raph@kali)-[/opt/lampp/htdocs]
$ ls
applications.html  dashboard  img        Lab2_HTMLInjection  vulnerable_app  week2_php_lab
bitnami.css       favicon.ico index.php    test.php            webalizer

(raph@kali)-[/opt/lampp/htdocs]
$ cd Lab2_HTMLInjection

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ nano index.html

```

2. Inside my folder I Created two new files (index.html & server.php)

```

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ sudo nano index.html

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ sudo nano server.php

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$

```

```

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ rm -r index.html
rm: remove write-protected regular file 'index.html'? y
rm: cannot remove 'index.html': Permission denied

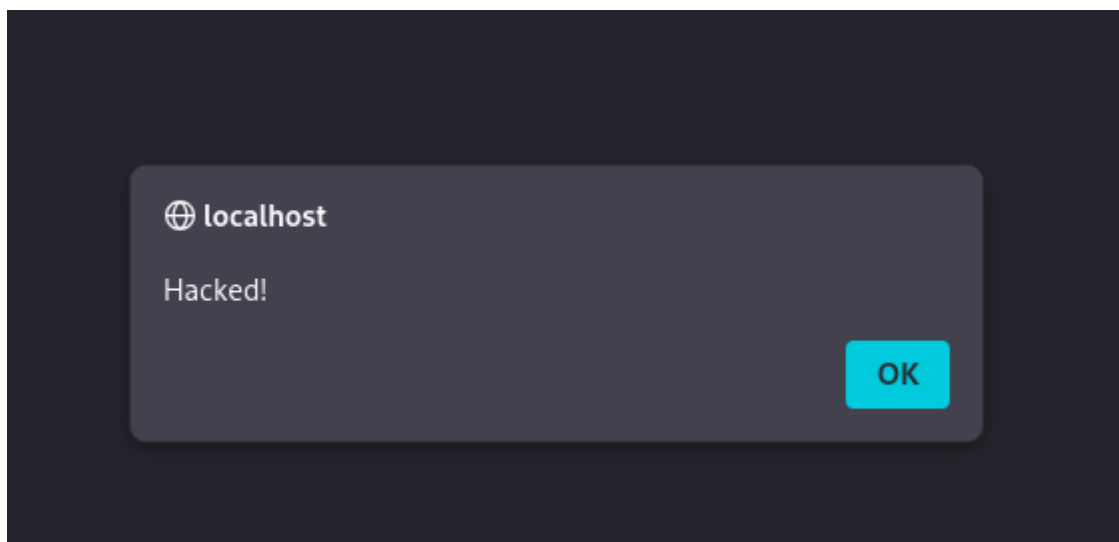
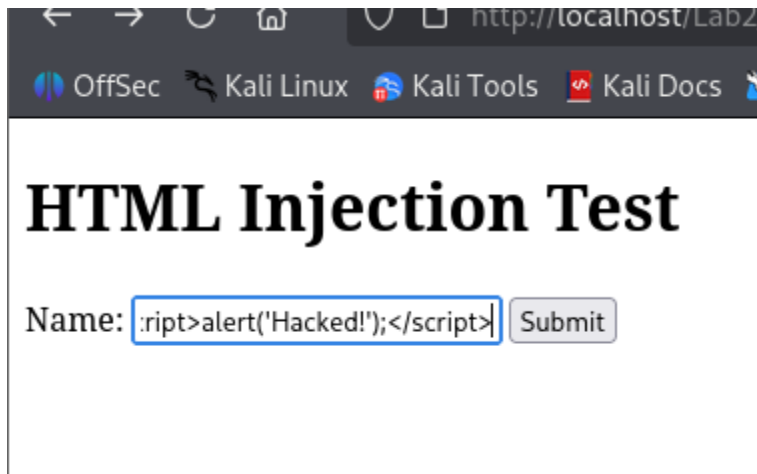
(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ rm -rf index.html
rm: cannot remove 'index.html': Permission denied

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ sudo rm -r index.html

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ sudo nano index.php

```

### Step 3: Exploit the Vulnerability



**Reflection Questions:**

- Why does this code allow the JavaScript to execute?

Because the website shows whatever the user types without checking it, the browser thinks the input is real code and runs it.

- What potential risks does this vulnerability pose in a real-world application?

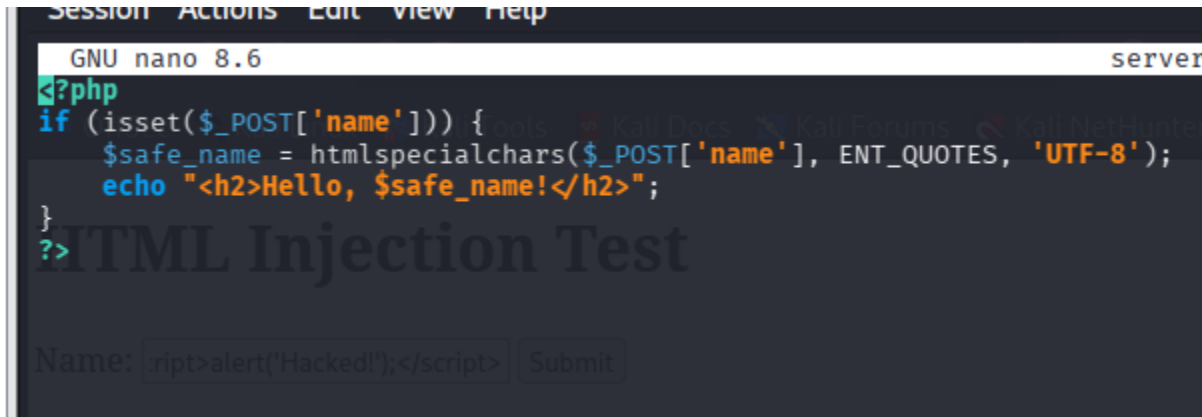
An attacker could use it to steal user information, mess with the website, or trick other users by running harmful code.

## Task 2: Mitigate HTML Injection Vulnerability

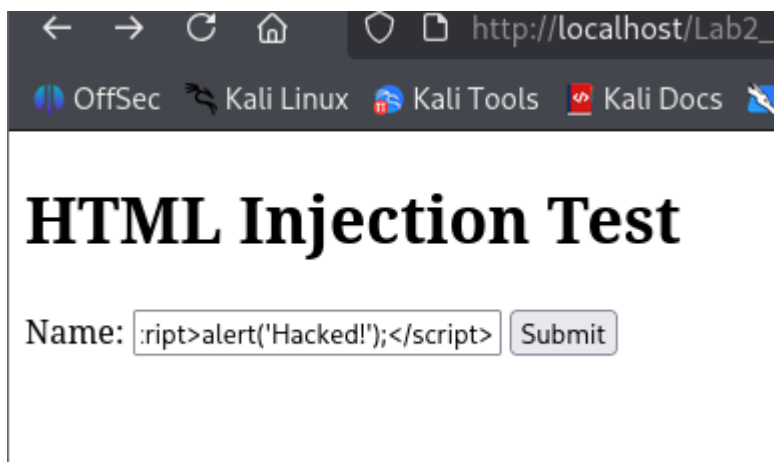
### Step 1: Fix the Vulnerability in server.php

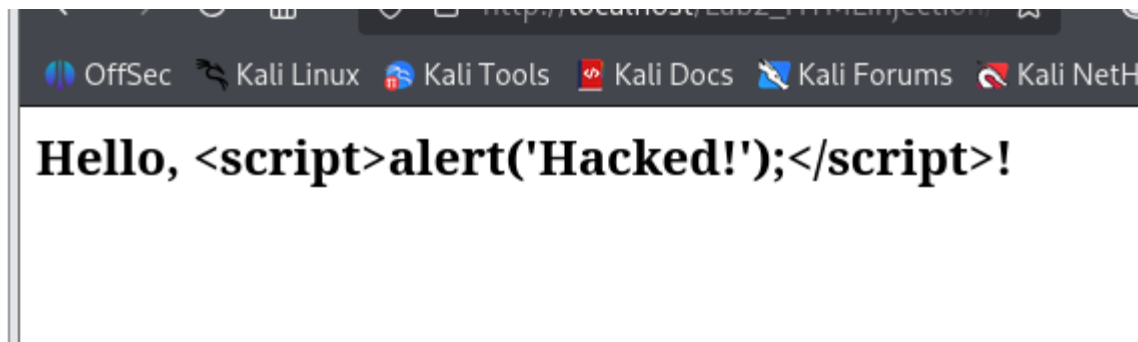
```
(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$ sudo nano server.php
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs/Lab2_HTMLInjection]
$
```



### Step 2: Test the Mitigated Version





### Reflection Questions:

- What happens to the malicious input after applying htmlspecialchars()?

The input is changed into plain text, so any special characters are shown on the page instead of being treated as code.

- How does this prevent the code from being executed?

Because the browser no longer sees the input as HTML or JavaScript, it does not run it.

- How do you feel about the importance of input sanitization in web security?

It is very important because it helps protect users and websites from attacks caused by unsafe user input.

### Deliverables

#### 1. Index.php

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>HTML Injection Example</title>
  </head>
  <body>
    <h1>HTML Injection Test</h1>
    <form action="server.php" method="post">
      <label for="name">Name:</label>
      <input type="text" id="name" name="name">
      <button type="submit">Submit</button>
    </form>

    <?php
    if (isset($_POST['name'])) {
      echo "<h2>Hello, " . $_POST['name'] . "!</h2>";
```

```
    }  
    ?>  
</body>  
</html>
```

## 2. Server.php

```
<?php  
if (isset($_POST['name'])) {  
    echo "<h2>Hello, " . $_POST['name'] . "!</h2>";  
}  
?>
```

## 3. Modify server.php

```
<?php  
if (isset($_POST['name'])) {  
    $safe_name = htmlspecialchars($_POST['name'], ENT_QUOTES, 'UTF-8');  
    echo "<h2>Hello, $safe_name!</h2>";  
}  
?>
```

## Lesson Learned

From this lab, I learned that accepting user input without checking it can make a web application unsafe. I saw how HTML injection can allow attackers to run unwanted code on a webpage. I also learned that using simple security functions like `htmlspecialchars()` can stop these attacks by turning harmful input into safe text.

## Conclusion

This lab helped me understand how HTML injection works and why input sanitization is important in web development. By testing both the vulnerable and fixed versions of the code, I gained practical knowledge on how to protect web applications. This experience showed me the importance of writing secure code to keep users and systems safe.